Where Protocol Fingerprints Stop: Seven Limits of Real TLS Fingerprinting Tools and a Stateful QUIC Initial Extractor

Miloni Thakkar	Suhani	Yash Goyal	Laxmidhar Panda
Roll No: 24110204	Roll No: 24110358	Roll No: 24110399	Roll No: 24110185

CS331 Computer Networks | Team T013 | Instructor: Dr. Sameer G. Kulkarni
Repository: https://github.com/milonithakkar/CS331-Phase2 (branch main)

I. Introduction

The first message of a TLS connection, the ClientHello, is sent unencrypted. Each TLS library fills it a little differently, so a hash of its fields (JA3, JA4) works as a fingerprint of the client software. JA4T does the same for the TCP SYN, JA4X for the server certificate and JA4H for the HTTP request. Networks use these fingerprints to recognise applications without decrypting traffic.

In Phase 1 we wrote our own fingerprinting code. For Phase 2 we used the tools that are used in practice, Wireshark/TShark (C) and FoxIO's ja4 (Rust), and measured where they stop giving a useful answer. We made real captures for seven situations (L1 to L7). We also wrote our own QUIC Initial extractor in C++ and compared it byte for byte with TShark. While doing this we found that a stateless extractor (keys from each packet's own connection ID) fails to decrypt the client packets sent after the server's connection ID switch, and we added a stateful mode that handles it.

II. Problem Statement

1. Protocol changes: QUIC wraps the ClientHello in encrypted Initial packets; TLS 1.3 encrypts the certificate; ECH hides the site name; post-quantum key shares make the ClientHello larger; browsers add GREASE values and shuffle extensions. Each change can break some fingerprint.

2. Capture point: TCP fingerprints (JA4T, JA4TS) read SYN values such as MSS, window and TTL. A proxy that ends the TCP connection and opens a new one replaces these values. Segmentation offload (TSO/GSO) changes how a large ClientHello appears in a capture.

3. QUIC connection ID switch (RFC 9000 §7.2, RFC 9001 §5.2): Initial keys are derived from the Destination Connection ID (DCID) of the client's first Initial. After the server replies, the client sends further Initials to the server's chosen connection ID, but the keys stay the same. An extractor that derives keys from each packet's own DCID cannot decrypt those packets.

Scope

In scope: seven limitation tests with TShark and FoxIO on real captures; QUIC v1 Initial extraction in C++; stateful association on a fixed UDP 4-tuple; damaged-capture tests; one paired TCP capture on Windows.

Out of scope: QUIC Retry, connection migration, QUIC v2, TLS 1.2, IPv6 (no IPv6 route available).

III. Project Objectives

1. Deliverable A (landscape): for JA3, JA3S, JA4, JA4S, JA4T, JA4X and JA4H, list the input fields, the normalisation and the RFC sections (docs/landscape.md).

2. Deliverable B (seven limitations): for each of L1 to L7, one test that changes one variable, with real captures, TShark and FoxIO output, and what the test does not prove (docs/tests/, LEDGER.md).

3. Deliverable C (depth dives): a C++20 QUIC Initial extractor checked against TShark, with a stateful mode for the connection ID switch; the extractor on damaged copies of the captures (reordered, dropped, truncated packets); a paired capture of the same TCP connection inside a container and on the Wi-Fi card.

IV. Architecture and Design

The extractor is a chain of small steps, one C++ file each:

```
pcap
 -> capture_reader   Ethernet, IPv4/6, UDP
 -> packet_parser    QUIC header, DCID, SCID
 -> initial_context  stateful mode only
 -> initial_crypto   keys, header protection,
                     AES-128-GCM
 -> frame_parser + crypto_reassembly
 -> client_hello     ClientHello bytes
```

Design choices

Small state: the stateful mode keeps, per UDP 4-tuple, only the first client DCID, the client SCIDs and the server SCIDs. It does not track the whole connection.

Few dependencies: C++20 and OpenSSL 3 only. The C++ tests are one test program run by CTest.

No JA4 in C++: the extractor outputs the ClientHello bytes. JA4 strings are computed by FoxIO's tool, so we do not maintain a second JA4 implementation.

V. Implementation

capture_reader.cpp: reads classic pcap files, returns UDP payloads from unfragmented IPv4/IPv6.

packet_parser.cpp: parses QUIC long headers. **initial_context.cpp:** InitialContextAssociator::observe(), the stateful mode. **initial_crypto.cpp:** Initial secrets (RFC 9001 §5.2), header protection (§5.4), AES-128-GCM decryption. **frame_parser.cpp, crypto_reassembly.cpp:** CRYPTO frames joined by offset. **client_hello.cpp:** checks the ClientHello structure, reads SNI, ciphers and extensions.

Stateful mode

```
flow = (client ip:port, server ip:port)
client Initial:
  first on flow -> its DCID = key source
  DCID = key source or server SCID
                -> decrypt with those keys
  other DCID    -> not associated
server Initial (DCID = client SCID):
  remember its SCID as an alias
```

Python scripts in code/scripts/ run the Track 1 captures and analysis, the damaged-capture tests and the verifiers. Key derivation is checked against RFC 9001 Appendix A.

VI. Testbed Setup

Track 1 captures (L1 to L7): Docker image cn-lab (Debian trixie) on a MacBook (aarch64), campus network. TShark 4.4.19, FoxIO ja4 tag v0.18.6 (reports 0.18.5), FoxIO main (commit c6469a7), Chromium 156, curl 8.14.1 with OpenSSL 3.5.7 (QUIC via OpenSSL), Python 3.13.5. Versions in results/versions.txt.

Track 2 batch and L3 paired capture: Windows 11 laptop (Intel Core Ultra 7 256V), WSL2 Ubuntu (kernel 6.18.33.2), GCC 15.2.0, OpenSSL 3.5.5, Python 3.14.4, TShark 4.6.4, Docker Desktop. Capture points: container eth0, WSL eth0, Windows vEthernet (WSL) and the Wi-Fi card.

Re-check (6 Nov): TShark 4.6.8 on the Mac and FoxIO tag v1.0.1, on the same committed pcaps (results/track1/recheck-20261106.md).

VII. Evaluation and Results

1. Seven limitations (Track 1)

Row	Question	Result	Status
L1	Can tools read the QUIC ClientHello?	TShark and FoxIO: 5/5 for curl and Chromium; Chromium's hello is in 21 CRYPTO frames	Guess wrong: they can
L2	Which fingerprints need TLS keys?	JA4X + JA4H: 0 without keys, 5 with keys (one TLS 1.3 session)	Confirmed
L3	Does the capture point matter?	Offload on: hello in 1 frame of 1586 B; off: 1514 + 138 B. Container SYN-ACK MSS 65495, TTL 63, from Docker's proxy	Confirmed
L4	Post-quantum key share	Hello 508 B (X25519) vs 1511 B (X25519MLKEM768); JA4 14 vs 12 extensions	Confirmed
L5	One browser = one fingerprint?	Chromium, 50 hellos: 50 JA3. JA4: 15 with TShark 4.4.19/4.6.8 and FoxIO v0.18.6; 1 with FoxIO v1.0.1	Confirmed
L6	IPv4 vs IPv6	IPv6 capture has 0 packets	Not measured
L7	Does ECH hide the site?	ECH on: SNI cloudflare-ech.com, but plain DNS asks for crypto.cloudflare.com. DoH run did not load	Confirmed; DoH not measured

L4: the size change comes from the key share: 1216 B (X25519 + ML-KEM-768) instead of 32 B. The JA4 change is a side effect: the PQ-only hello drops ec_point_formats, and the small classic hello gets a padding extension.

L5: Chromium puts a random GREASE value first in signature_algorithms; the 50 hellos use 15 different values. FoxIO stopped counting GREASE there in commit d66336e, first released in v1.0.1 (31 Aug 2026). So the JA4 of one browser depends on the tool version used.

2. QUIC extractor, captured order

WSL batch verification-batch-20261105-105416, compared with TShark 4.6.4 on the same file.

	curl	Chromium
UDP 4-tuples	5	5
Client Initial packets	20	16
Stateless: decrypted / failed	10 / 10	10 / 6
Stateful: decrypted / failed	20 / 0	16 / 0
ClientHellos, C++ / TShark	5 / 5	5 / 5
Same bytes as TShark	yes	yes

The failed packets are the client Initials sent to the server's connection ID. TShark already decrypts them; the stateful mode makes our extractor match it. In these two captures those packets contain only ACK and PADDING frames, so the ClientHello, and the fingerprint, is the same in both modes. The stateful mode matters when part of the ClientHello is sent again after the server's reply.

3. Damaged copies of the two QUIC captures

ClientHellos matched / TShark ClientHellos, per capture (same for curl and Chromium):

Damage	Stateless	Stateful
none	5 / 5	5 / 5
reverse packet order	5 / 5	0 / 5

shuffled order	5 / 5	4 / 5
one packet removed	5 / 5	5 / 5
every third packet removed	2 / 2	2 / 2
one packet cut by 64 bytes	5 / 5	5 / 5

Stateless mode matched all 54 TShark ClientHellos; stateful mode matched 42 of 54. When a later client packet comes first, the stateful mode takes its connection ID as the key source and then rejects the real first packet. With every third packet removed, TShark also finds only 2 complete ClientHellos per capture.

4. Paired TCP capture on Windows

The same three requests were captured inside the Docker container and on the Wi-Fi card at the same time (results/l3_paired_batch/). Client SYN values:

Capture point	TTL	Window	Wscale	MSS	Timestamps
container eth0	64	64240	10	1460	yes (Linux)
Wi-Fi card	128	65535	8	1460	no (Windows)

The ISNs differ, so the Wi-Fi SYN belongs to a new TCP connection made by Windows. A JA4T taken on the Wi-Fi describes Windows, not the Linux container. In the same capture, traffic from a Linux VM on the vEthernet (WSL) adapter keeps its ISN and Linux values on the Wi-Fi, with TTL 64 -> 63 (normal NAT). The container saw a SYN-ACK with MSS 1460 and wscale 7, while the server's SYN-ACK on the Wi-Fi had MSS 1400 and wscale 13, so the container's SYN-ACK also came from a local stack.

VIII. Summary

Real tools read QUIC ClientHellos without trouble (L1). They lose JA4X and JA4H without TLS keys (L2). TCP fingerprints describe the nearest TCP stack, which in our labs was Docker's proxy or the Windows host (L3). A post-quantum key share triples the ClientHello size and changes JA4 through side effects (L4). One Chromium gives 15 JA4s with TShark 4.6.8 and older FoxIO but 1 with FoxIO v1.0.1, so fingerprint lists must record the tool version (L5). ECH hides the name in TLS but plain DNS still shows it (L7). Our C++ extractor gives the same ClientHello bytes as TShark; its stateful mode decrypts all client Initials (+10 curl, +6 Chromium packets) but is sensitive to packet order.

Limitations

Track 1 captures were taken inside Docker on one Mac on one network; no IPv6; DoH did not work on campus. One site (Cloudflare) and three clients (Chromium, curl, Python). The stateful mode fails when post-switch packets come before the first packet; no Retry, migration or QUIC v2.

Future work

1. In the stateful mode, fall back to the packet's own DCID when it is unknown, so reordering does not lose ClientHellos. 2. QUIC Retry handling. 3. Repeat L3 and L6 on a plain Linux machine with IPv6, outside Docker.

IX. Reproducibility

```
git clone https://github.com/
    milonithakkar/CS331-Phase2.git
cd CS331-Phase2
make test    # CTest 1/1, pytest 52
make check   # 16 pcaps match SHA-256
python3 code/scripts/\
    verify_all_seven_rows.py
             # 6/7 PASS, L6 NOT_MEASURED
make lab-image && make lab-eval
```

Batch results are in results/track2/, the Track 1 re-check in results/track1/recheck-20261106.md.

References

[1] M. Thomson and S. Turner, "Using TLS to Secure QUIC," RFC 9001, May 2021.

[2] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, May 2021.

[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018.

[4] D. Benjamin, "Applying GREASE to TLS Extensibility," RFC 8701, Jan. 2020.

[5] FoxIO, "JA4+ Network Fingerprinting," tags v0.18.6 and v1.0.1. https://github.com/FoxIO-LLC/ja4

[6] Wireshark Foundation, "Wireshark / TShark," versions 4.4.19, 4.6.4, 4.6.8. https://www.wireshark.org

[7] E. Rescorla, K. Oku, N. Sullivan, C. A. Wood, "TLS Encrypted Client Hello," draft-ietf-tls-esni.

[8] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Aug. 2022.

Appendix A: Team Work Distribution and Member Rating

#	Member	Roll No.	Tasks done	Rating
1	Miloni Thakkar	24110204	Project technical lead: co-wrote the Phase 2 plan; designed and wrote the C++20 QUIC extractor and InitialContextAssociator; fixed the post-switch decryption failures; debugged the Track 1 JSON baseline parser; built the WSL testbed (pytest 52, CTest 1/1); ran the Windows/WSL paired captures; wrote the report and maintained the repository.	10
2	Suhani	24110358	Architecture co-design and	9

			verification: co-wrote the Phase 2 plan and design decisions; set up verification steps to find baseline/capture defects; wrote Deliverable A (JA3/JA4/JA4T/JA4X/JA4H landscape); documented field mappings in docs/basics/.	
3	Yash Goyal	24110399	Lab captures and tooling: built the Docker lab (cn-lab) with pinned FoxIO ja4 and TShark 4.4; made the Track 1 captures (curl, Chromium, Python; QUIC, keys, offload, PQ, repeats, ECH); wrote the Track 1 capture and analysis scripts and test notes.	8
4	Laxmidhar Panda	24110185	L3 transport analysis and presentation: helped with the L3 paired capture; reviewed the TCP values (MSS, TTL, window scale); structured the presentation; helped with report formatting.	7

Appendix B: AI Usage

All prompts and transcripts are kept unedited in the repository's ai/ folder.

#	AI tool	Used for	Record
1	Google Antigravity	C++ pair programming, TShark JSON parser debugging, InitialContextAssociator, WSL batch scripts, report formatting	ai/miloni_(antigravity).md
2	Arena agent	Repository review, consistency checks of claims and evidence	ai/miloni_(arena).md
3	Claude Code (Claude Opus 5.5)	Track 1 Docker lab, capture and analysis scripts, test notes T0 to T7, 6 Nov re-check of Track 1 numbers, final consistency review of report, slides and docs	ai/yash_(claude).md